СОДЕРЖАНИЕ

1. Полезные советы 2
2. Хранение результатов и вывод 2
3. Простые числа 3
   3.1. Советы по решению 3
   3.2. Функция проверки на простое число 3
   3.3. Функция для составления решета Эратосфена 3
4. Модульная арифметика 5
   4.1. Теория 5
   4.2. Советы по решению 5

1. Полезные советы

1. Писать условие и рассуждения на бумаге
2. Хранить результаты сразу в *long long* вместо *int*

2. Хранение результатов и вывод

1. Не хранить результат в переменных/массиве/строке, когда её надо будет сразу выводить. Можно просто выводить сразу.
2. Вместо поиска всех решений достаточно понять, можно ли выводить лишь одно для определённых случаев

3. Простые числа

3.1. Советы по решению

1. Если нужно найти простое число в числе, то достаточно искать лишь два простых числа (зеркальных, по типу 13 и 31).
2. Любое число можно разделить на ПРОСТОЕ ЧИСЛО НАЦЕЛО, кроме 1.
3. Любое число можно разложить на простые множители, кроме 1.
4. Забрать больше всего простых чисел посредством префиксных сумм из мн-ва $\{1, 2\}$ можно, начиная с тройки и продолжать прибавлять двойки, пока не закончатся (пр.: *2 1 2 2 2 2 1 1 1 = 2 3 5 7 9 11 12 13 14*.
5. Чтобы получить из ДВУХ ПРОСТЫХ чисел НЕ ПРОСТОЕ ЧИСЛО, достаточно:

$$(p \neq 2 \in P) + (p = 3 \text{ (or any prime number)} \in P) = (n \notin P)$$

и

$$(p = 2 \in P) + (p = 2 \text{ (or any even number)} \in P) = (n \notin P).$$

3.2. Функция проверки на простое число

```
bool checkPrime(int n) {
  if (n == 0 or n == 1) return false;
  for (int i = 0; i * i <= n; i++)
    if (n % i == 0)
      return false;
  return true;
}
```

3.3. Функция для составления решета Эратосфена

```
#include <iostream>
#include <vector>
using namespace std;
vector<int> sieve(int n) {
  vector<int> primes(n + 1, true);
  primes[0] = false;
  primes[1] = false;
  for (int i = 2; i * i <= n; i++)
    if (primes[i])
      for (int j = i * i; j <= n; j += i)
        primes[j] = false;
  return primes;
}
int main() {
```

```
  int n;
  cin >> n;
  vector<int> primes = sieve(n);
  for (int i = 0; i <= n; i++)
    if (primes[i]) cout << i << " is a primer." << endl;
    else cout << i << " isn't a primer." << endl;
  return 0;
}
```

4. Модульная арифметика

4.1. Теория

$$(A + B)\%m = ((A\%m) + (B\%m))\%m,$$

$$(A - B)\%m = ((A\%m) - (B\%m))\%m,$$

$$(A * B)\%m = ((A\%m) * (B\%m))\%m,$$

где $(A, B, m \in \mathbb{N})$.

4.2. Советы по решению